```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
from google.colab import files

uploaded = files.upload()
```

Choose Files Mall_Customers.csv

Mall_Customers.csv(text/csv) - 3981 bytes, last modified: 9/21/2026 - 100% done

```
Saving Mall_Customers.csv to Mall_Customers.csv
```

```
import pandas as pd

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

print("Dataset loaded successfully!")
print("Shape:", df.shape)

display(df.head())
```

```
Dataset loaded successfully!
Shape: (200, 5)
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
print("Column Names:")
print(df.columns.tolist())
```

```
Column Names:
['CustomerID', 'Gender', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)']
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

```
df.describe()
```

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
print("Missing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
```

```
df = df.dropna()

print("Shape after removing missing values:", df.shape)
```

```
Shape after removing missing values: (200, 5)
```

```
print("Number of duplicate rows:", df.duplicated().sum())
```

```
Number of duplicate rows: 0
```

```
features = ['Age', 'Annual Income (k$)', 'Spending Score (1-100)']

X = df[features]

display(X.head())
```

	Age	Annual Income (k$)	Spending Score (1-100)
0	19	15	39
1	21	15	81
2	20	16	6
3	23	16	77
4	31	17	40

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("Feature scaling completed!")
print(X_scaled[:5])
```

```
Feature scaling completed!
[[-1.42456879 -1.73899919 -0.43480148]
 [-1.28103541 -1.73899919  1.19570407]
 [-1.3528021  -1.70082976 -1.71591298]
 [-1.13750203 -1.70082976  1.04041783]
 [-0.56336851 -1.66266033 -0.39597992]]
```

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

wcss = []
```

```
for k in range(1, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

print("WCSS values:")
print(wcss)
```

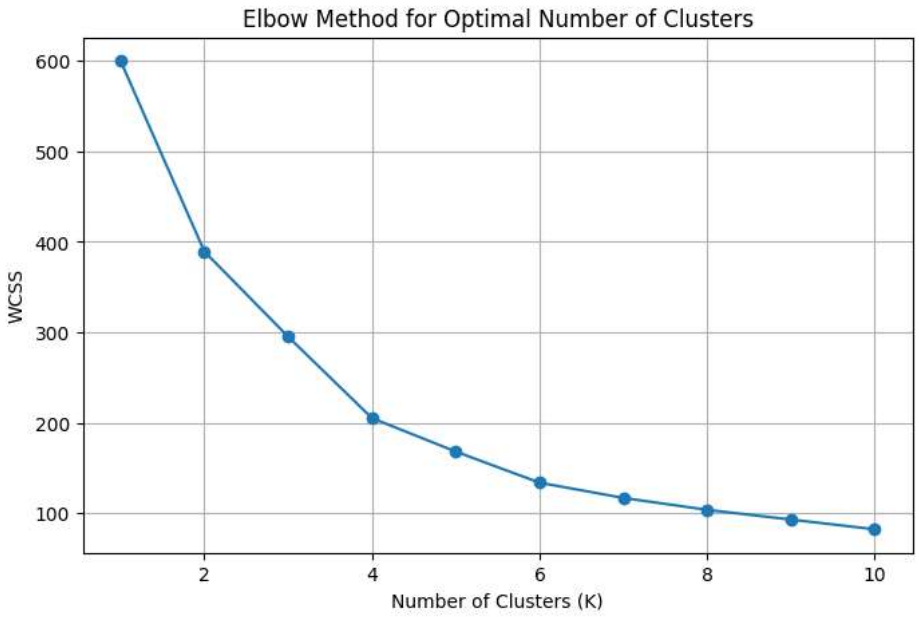
```
WCSS values:
[599.9999999999999, 389.38618895643714, 295.21224615554877, 205.22514747675913, 168.2475801755683, 133.86842085478855, 117.01155
```

```
plt.figure(figsize=(8, 5))

plt.plot(
    range(1, 11),
    wcss,
    marker='o'
)

plt.title("Elbow Method for Optimal Number of Clusters")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.grid(True)

plt.show()
```




```
from sklearn.metrics import silhouette_score

silhouette_scores = []

for k in range(2, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = kmeans.fit_predict(X_scaled)

    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)

    print(f"K = {k}, Silhouette Score = {score:.4f}")
```

```
K = 2, Silhouette Score = 0.3355
K = 3, Silhouette Score = 0.3578
K = 4, Silhouette Score = 0.4040
K = 5, Silhouette Score = 0.4166
K = 6, Silhouette Score = 0.4284
K = 7, Silhouette Score = 0.4172
K = 8, Silhouette Score = 0.4082
K = 9, Silhouette Score = 0.4177
K = 10, Silhouette Score = 0.4066
```

```
plt.figure(figsize=(8, 5))

plt.plot(
    range(2, 11),
    silhouette_scores,
    marker='o'
)

plt.title("Silhouette Score for Different K Values")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.grid(True)

plt.show()
```




```
best_k = range(2, 11)[np.argmax(silhouette_scores)]

print("Best K according to Silhouette Score:", best_k)
```

```
Best K according to Silhouette Score: 6
```

```
kmeans = KMeans(
    n_clusters=best_k,
    random_state=42,
    n_init=10
)

df['Cluster'] = kmeans.fit_predict(X_scaled)

print("K-Means clustering completed successfully!")

display(df.head())
```

```
K-Means clustering completed successfully!
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	4
2	3	Female	20	16	6	5
3	4	Female	23	16	77	4
4	5	Female	31	17	40	5

```
print("Number of customers in each cluster:")

cluster_counts = df['Cluster'].value_counts().sort_index()

display(cluster_counts)
```

```
Number of customers in each cluster:
```

Cluster	count
0	45
1	39
2	33
3	39
4	23
5	21

dtype: int64

```
cluster_centers = scaler.inverse_transform(
    kmeans.cluster_centers_
)

centers_df = pd.DataFrame(
    cluster_centers,
    columns=features
)

centers_df.index.name = "Cluster"

print("Cluster Centers:")

display(centers_df)
```

```
Cluster Centers:
```

Cluster	Age	Annual Income (k$)	Spending Score (1-100)
0	56.333333	54.266667	49.066667
1	26.794872	57.102564	48.128205
2	41.939394	88.939394	16.969697
3	32.692308	86.538462	82.128205
4	25.000000	25.260870	77.608696
5	45.523810	26.285714	19.380952

Next steps: Generate code with `centers_df` New interactive sheet

```
plt.figure(figsize=(10, 7))

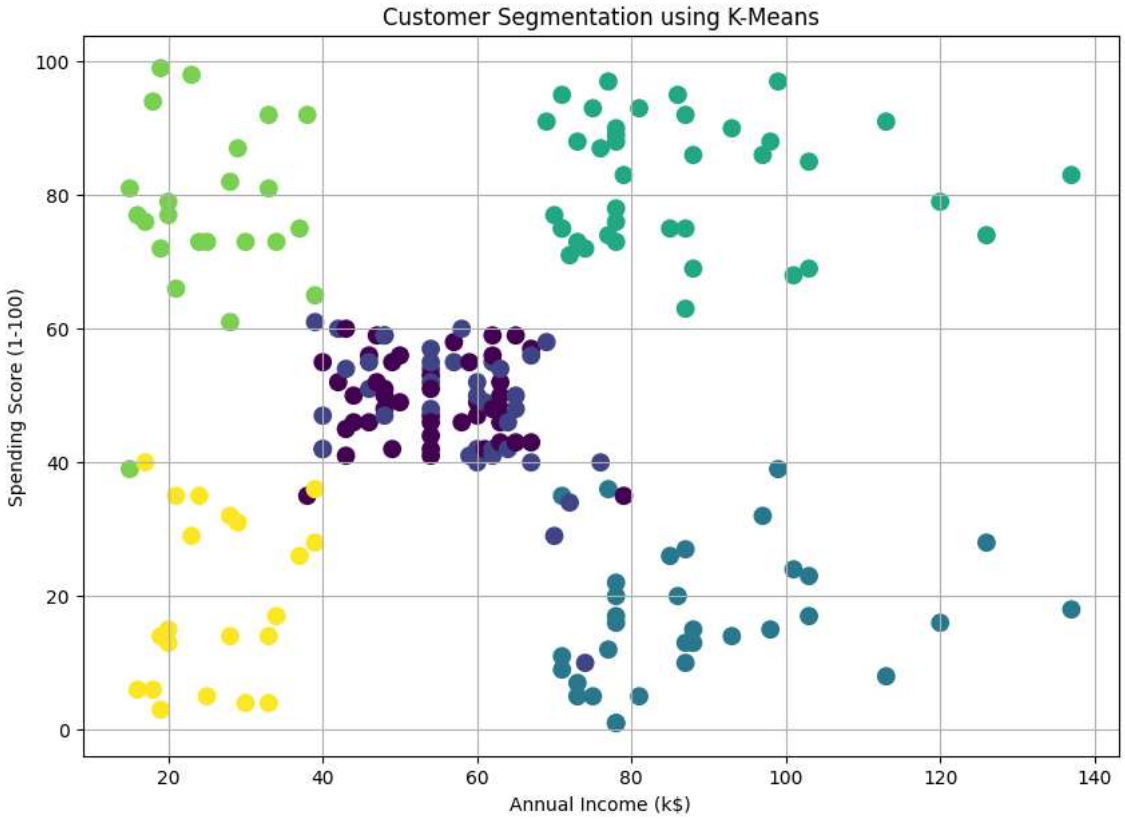
plt.scatter(
    df['Annual Income (k$)'],
    df['Spending Score (1-100)'],
    c=df['Cluster'],
    s=80
```

```
)

plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title('Customer Segmentation using K-Means')

plt.grid(True)

plt.show()
```



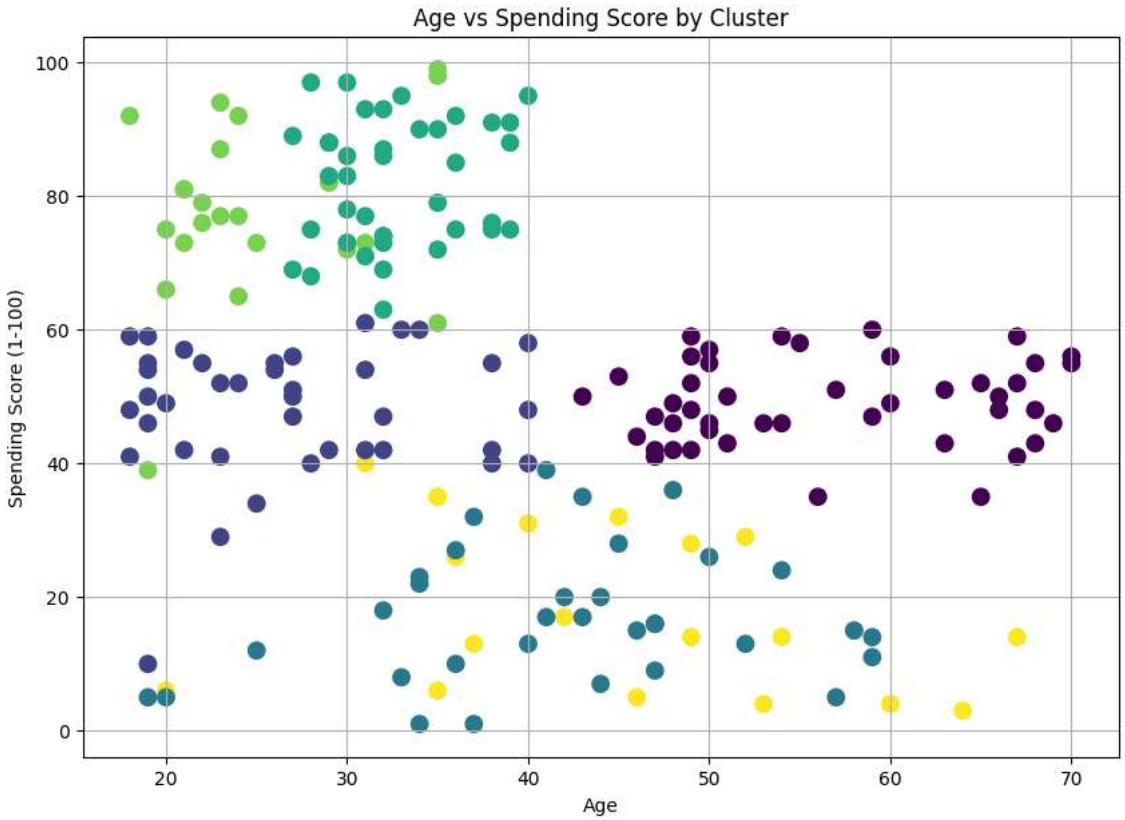

```
plt.figure(figsize=(10, 7))

plt.scatter(
    df['Age'],
    df['Spending Score (1-100)'],
    c=df['Cluster'],
    s=80
)

plt.xlabel('Age')
plt.ylabel('Spending Score (1-100)')
plt.title('Age vs Spending Score by Cluster')

plt.grid(True)

plt.show()
```



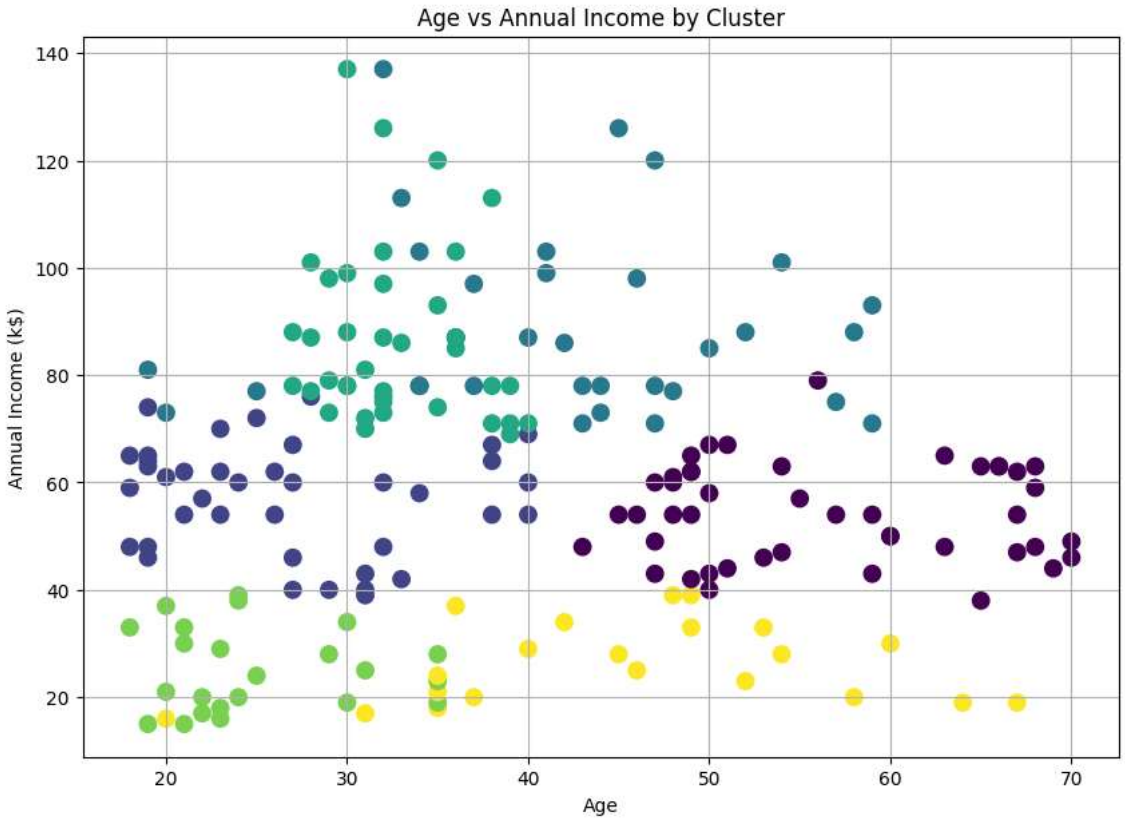

```
plt.figure(figsize=(10, 7))

plt.scatter(
    df['Age'],
    df['Annual Income (k$)'],
    c=df['Cluster'],
    s=80
)

plt.xlabel('Age')
plt.ylabel('Annual Income (k$)')
plt.title('Age vs Annual Income by Cluster')

plt.grid(True)

plt.show()
```




```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

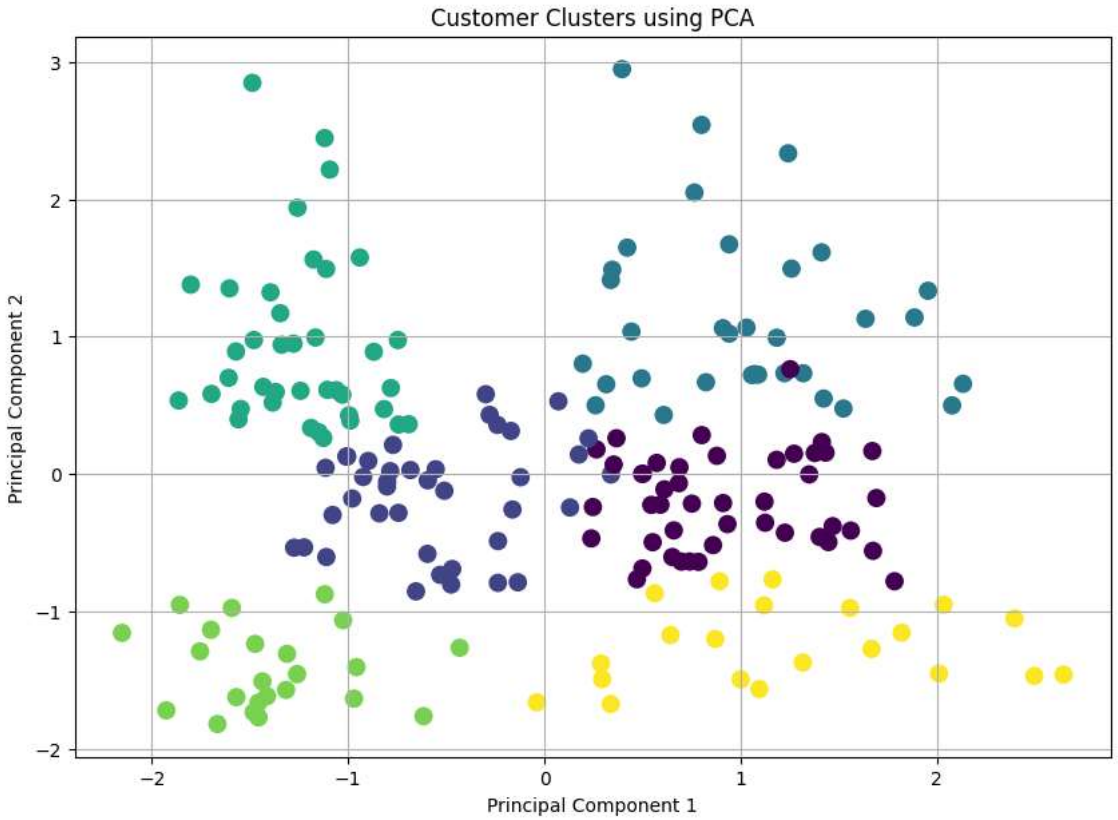
plt.figure(figsize=(10, 7))

plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=df['Cluster'],
    s=80
)

plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('Customer Clusters using PCA')

plt.grid(True)

plt.show()
```




```
cluster_summary = df.groupby('Cluster')[features].mean()

print("Average characteristics of each customer cluster:")

display(cluster_summary)
```

Average characteristics of each customer cluster:

Cluster	Age	Annual Income (k$)	Spending Score (1-100)
0	56.333333	54.266667	49.066667
1	26.794872	57.102564	48.128205
2	41.939394	88.939394	16.969697
3	32.692308	86.538462	82.128205
4	25.000000	25.260870	77.608696
5	45.523810	26.285714	19.380952

Next steps: Generate code with `cluster_summary` New interactive sheet

```
summary = df.groupby('Cluster').agg({
    'Age': 'mean',
    'Annual Income (k$)': 'mean',
    'Spending Score (1-100)': 'mean',
    'CustomerID': 'count'
})

summary.rename(
    columns={
        'CustomerID': 'Number of Customers'
    },
    inplace=True
)

summary = summary.round(2)

display(summary)
```

Cluster	Age	Annual Income (k$)	Spending Score (1-100)	Number of Customers
0	56.33	54.27	49.07	45
1	26.79	57.10	48.13	39
2	41.94	88.94	16.97	33
3	32.69	86.54	82.13	39
4	25.00	25.26	77.61	23
5	45.52	26.29	19.38	21

Next steps: Generate code with summary | New interactive sheet

```
for cluster in sorted(df['Cluster'].unique()):

    data = df[df['Cluster'] == cluster]

    avg_age = data['Age'].mean()
    avg_income = data['Annual Income (k$)'].mean()
    avg_spending = data['Spending Score (1-100)'].mean()

    print(f"\nCluster {cluster}")
    print("-" * 40)

    print(f"Number of Customers : {len(data)}")
    print(f"Average Age         : {avg_age:.2f}")
    print(f"Average Income      : {avg_income:.2f} k$")
    print(f"Average Spending    : {avg_spending:.2f}")
```

```
Cluster 0
----------------------------------------
Number of Customers : 45
Average Age         : 56.33
Average Income      : 54.27 k$
Average Spending    : 49.07

Cluster 1
----------------------------------------
Number of Customers : 39
Average Age         : 26.79
Average Income      : 57.10 k$
Average Spending    : 48.13

Cluster 2
----------------------------------------
Number of Customers : 33
Average Age         : 41.94
Average Income      : 88.94 k$
Average Spending    : 16.97

Cluster 3
----------------------------------------
Number of Customers : 39
Average Age         : 32.69
Average Income      : 86.54 k$
Average Spending    : 82.13

Cluster 4
----------------------------------------
Number of Customers : 23
Average Age         : 25.00
Average Income      : 25.26 k$
Average Spending    : 77.61

Cluster 5
----------------------------------------
Number of Customers : 21
Average Age         : 45.52
Average Income      : 26.29 k$
Average Spending    : 19.38
```

Part F — Marketing Strategy

Customer Segmentation and Marketing Strategies

Customer segmentation helps businesses divide customers into groups based on their characteristics and purchasing behavior. Different strategies can be applied to different customer segments.

1. High Income + High Spending Score

These customers have relatively high purchasing power and high spending behavior. Businesses can target them with premium products, exclusive offers, loyalty programs, and personalized recommendations.

2. High Income + Low Spending Score

These customers have relatively high purchasing power but comparatively lower spending behavior. Businesses can use personalized offers, attractive promotions, product recommendations, and special discounts to encourage them to purchase more.

3. Low Income + High Spending Score

These customers have relatively lower income but demonstrate comparatively high spending behavior. Businesses can provide affordable products, discounts, loyalty rewards, and attractive value-based offers.

4. Low Income + Low Spending Score

These customers have relatively lower income and lower spending behavior. Businesses can target them with budget-friendly products, discounts, promotional offers, and value-for-money products.

Conclusion of Marketing Analysis

The identified customer segments can help businesses design targeted marketing campaigns. By understanding the income and spending behavior of different customer groups, businesses can provide more relevant offers, improve customer engagement, increase customer satisfaction, and support customer retention.

```
final_silhouette = silhouette_score(
    X_scaled,
    df['Cluster']
)

print("Final Silhouette Score:", round(final_silhouette, 4))
```

```
Final Silhouette Score: 0.4284
```

```
df.to_csv(
    "customer_segmentation_result.csv",
    index=False
)

print("Final clustered dataset saved successfully!")
```

```
Final clustered dataset saved successfully!
```

```
from google.colab import files

files.download("customer_segmentation_result.csv")
```

Theory Questions and Answers

Q1. What is clustering? How does it differ from classification?

Answer:

Clustering is an unsupervised machine learning technique used to group similar data points into different groups called clusters. It does not require predefined class labels.

Classification is a supervised machine learning technique in which a model learns from labeled data and assigns new data to predefined classes.

Difference:

Clustering:

- It is an unsupervised learning technique.
- It does not require predefined labels.
- It discovers natural groups in the data.
- Example: Customer segmentation.

Classification:

- It is a supervised learning technique.
- It uses predefined labels.
- It predicts the class of new observations.
- Example: Spam email classification.

Q2. Explain the working principle of the K-Means Clustering algorithm.

Answer:

K-Means is a clustering algorithm that divides data into K groups or clusters.

The main steps are:

1. Select the number of clusters K.
2. Initialize K cluster centroids.
3. Assign each data point to the nearest centroid.
4. Calculate the new centroid of each cluster.
5. Repeat the assignment and centroid calculation steps.
6. Continue until the centroids become stable or the stopping condition is reached.

The final result contains K clusters in which data points have similar characteristics.

Q3. Why is feature scaling important before applying K-Means clustering?

Answer:

Feature scaling is important because K-Means uses distance calculations to assign data points to clusters.

If one feature has a much larger numerical range than another feature, it can dominate the distance calculation.

For example, annual income may have values in thousands while age may have values between approximately 18 and 70.

Standardization puts the features on comparable scales so that each feature can contribute more equally to the clustering process.

Q4. What is the Elbow Method, and how does it help determine the optimal number of clusters?

Answer:

The Elbow Method is used to determine a suitable number of clusters for the K-Means algorithm.

It calculates the Within-Cluster Sum of Squares (WCSS) for different values of K.

As the value of K increases, WCSS generally decreases. The WCSS values are plotted against the number of clusters.

The point where the decrease in WCSS starts becoming slower is called the elbow point.

The elbow point can be selected as a suitable value for the number of clusters.

Q5. What is the Silhouette Score? How is it used to evaluate clustering performance?

Answer:

The Silhouette Score is a measure used to evaluate the quality of clustering.

Its value generally ranges from -1 to +1.

- A value close to +1 indicates that the data points are well separated from other clusters.
- A value close to 0 indicates that the clusters may overlap.
- A negative value may indicate that some data points have been assigned to an inappropriate cluster.

The Silhouette Score can be calculated for different values of K and used to evaluate the clustering result.

Q6. Why is K-Means considered an unsupervised machine learning algorithm?

Answer:

K-Means is considered an unsupervised machine learning algorithm because it does not require predefined class labels.

The algorithm receives input features and automatically discovers groups based on similarities and distances between data points.

In customer segmentation, customers can be grouped according to characteristics such as age, annual income, and spending score without providing predefined customer categories.

Q7. Mention any four real-world applications of customer segmentation.

Answer:

Four real-world applications of customer segmentation are:

1. Targeted marketing campaigns.
2. Personalized product recommendations.
3. Customer retention strategies.
4. Personalized discounts and offers.

Customer segmentation helps organizations identify groups of customers with similar characteristics and purchasing behavior.

Q8. What are the limitations of the K-Means algorithm?